Battery Arbitrage

Software Architecture and Module Responsibilities

Project Architecture Overview

github.com/Evnsan/battery-arbitrage

November 2026

Contents

1 Overview **2**

2 High-Level Architecture **2**

3 Module Responsibilities **3**

3.1 `cli.py` – Command-Line Interface 3

3.2 `pipeline.py` – Application Orchestration 3

3.3 `parsing.py` – Input Parsing 3

3.4 `validation.py` – Market Data Validation 4

3.5 `time_alignment.py` – Common Timeline Construction 4

3.6 `time_filter.py` – Optimization Period Selection 4

3.7 `domain.py` – Domain Data Structures 5

3.8 `optimization.py` – MILP Formulation and Solver Integration 5

3.9 `reporting.py` – Report Construction 5

3.10 `csv_export.py` – Report Persistence 6

3.11 `__init__.py` 6

4 Data Flow and Component Interaction **6**

5 Temporal Alignment **6**

6 Testing Strategy **7**

7 Design Characteristics and Trade-offs **8**

7.1 Common Physical Timeline 8

7.2 Market Interval Commitments 8

7.3 Optimization and Reporting Separation 8

7.4 Explicit Domain Contracts 8

7.5 Configurable Battery Parameters 8

8 Current Scope and Limitations **9**

9 Summary **9**

1 Overview

The project is a Python application that optimizes battery dispatch across multiple wholesale electricity markets.

It accepts market prices from an Excel workbook, validates and aligns market time series, formulates a Mixed-Integer Linear Programming (MILP) model, and exports the resulting dispatch schedule and battery operation reports.

The architecture is organized as a processing pipeline. Each stage has a relatively narrow responsibility, while the domain module provides the data structures exchanged between stages.

The main design goals are:

- Separate input handling from domain validation and optimization.
- Support markets with different trading interval durations.
- Represent battery-wide physical constraints consistently across markets.
- Keep optimization logic independent from report formatting and file export.
- Make the processing stages independently testable.

2 High-Level Architecture

The application follows this execution flow:

1. The command-line interface receives the input workbook and execution parameters.
2. The pipeline coordinates parsing, validation, time alignment, optional period selection, optimization, and report construction.
3. The optimization module builds and solves the Pyomo MILP model using HiGHS.
4. The reporting module transforms the optimization result into report-oriented domain objects.
5. The CSV export module writes the reports and metadata to disk.

Figure 1 summarizes the main components and their relationships.

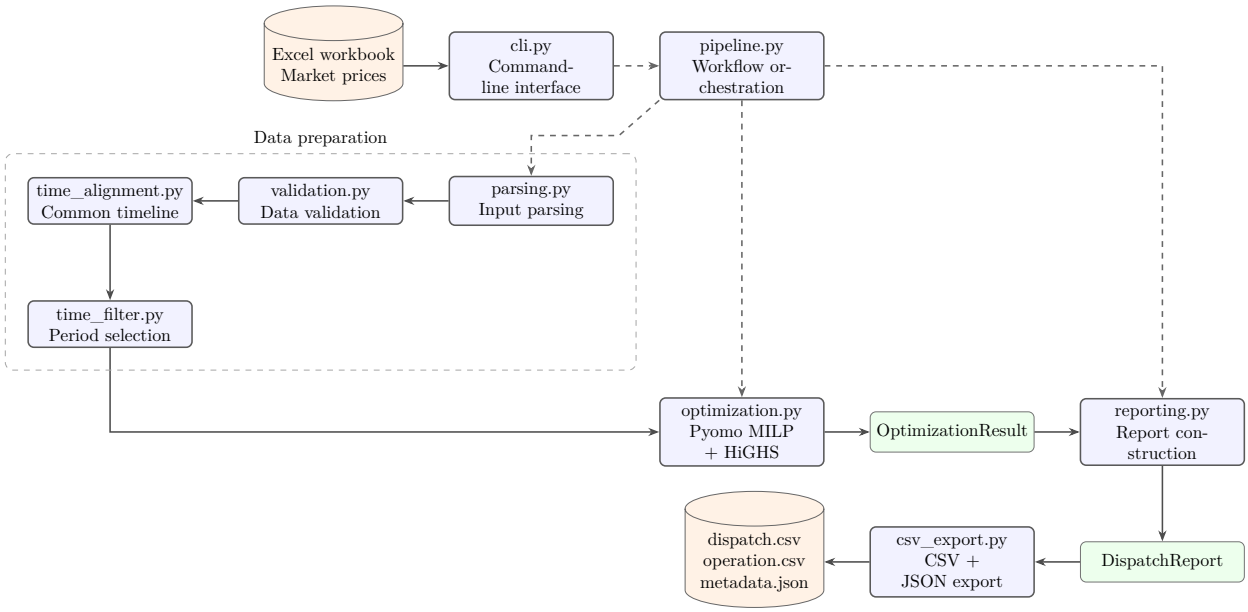


Figure 1: High-level application architecture and data flow.

In the diagram, dashed arrows show orchestration: the CLI invokes the pipeline, which coordinates data preparation, optimization, and report construction. Solid arrows show data moving between stages. The CLI also invokes the export functionality.

The `domain.py` module is not a processing stage. It provides shared data structures used by several components.

3 Module Responsibilities

The application modules are located under `src/battery_arbitrage/`.

3.1 `cli.py` – Command-Line Interface

Provides the executable entry point for the application.

Responsibilities:

- Defines and parses command-line arguments.
- Configures logging verbosity.
- Invokes the optimization pipeline.
- Determines the default output directory when one is not supplied.
- Calls the export module to persist the reports.
- Converts expected application errors into a non-zero process exit code.

The CLI does not implement data processing or optimization logic. It acts as the application's composition and execution boundary.

3.2 `pipeline.py` – Application Orchestration

Coordinates the complete processing workflow.

The `run_optimization` function executes the following stages:

1. Parse the Excel workbook.
2. Validate market timestamps and infer market intervals.
3. Align all markets to a common physical timeline.
4. Apply an optional optimization-period filter.
5. Optimize battery dispatch.
6. Build the final dispatch report.

It connects the individual modules without duplicating their responsibilities.

3.3 `parsing.py` – Input Parsing

Converts the Excel workbook into the application's market-data representation.

Responsibilities:

- Reads one worksheet per market.
- Identifies the price column using the `Price [£/MWh]` header suffix.
- Parses Excel datetime values and supported timestamp strings.
- Converts prices into floating-point values.
- Rejects missing, invalid, non-finite, or duplicate values.
- Ignores fully blank rows.

Its output is a mapping from market names to timestamp-price series.

The parser is intentionally unaware of battery configuration and optimization constraints.

3.4 `validation.py` – Market Data Validation

Checks the temporal consistency of each market's price series and infers its trading interval.

For each market, it:

- Sorts timestamps.
- Calculates consecutive timestamp differences.
- Infers the interval using the median difference.
- Checks that all consecutive differences are within the configured tolerance of the inferred interval.

The result is a mapping between market names and their inferred interval durations.

This stage establishes whether each market series has a consistent time frequency before alignment.

3.5 `time_alignment.py` – Common Timeline Construction

Transforms market-specific time series into a shared physical timeline.

This is an important part of the architecture because different markets can operate at different trading frequencies.

The module:

- Calculates the greatest common divisor of market intervals to determine the common timestep.
- Normalizes timestamps against a shared time grid.
- Applies the configured timestamp tolerance when snapping timestamps to the grid.
- Determines the time coverage shared by all markets.
- Repeats prices from longer market intervals across the physical timesteps they cover.
- Produces a `Timeline` containing aligned prices, the common timestep, and each market's original interval.

For example, an hourly market price is represented on two consecutive half-hour timesteps when the common physical resolution is 30 minutes.

The alignment stage does not make trading decisions. It creates a consistent temporal representation for the optimization model.

3.6 `time_filter.py` – Optimization Period Selection

Selects a subset of an already aligned timeline.

The module supports an inclusive start and exclusive end boundary:

$$[start, end)$$

It:

- Clips requested bounds to the available timeline coverage.
- Verifies that the selected period overlaps the available data.
- Checks timezone-awareness compatibility.
- Ensures that the selected boundaries align with boundaries shared by all markets.
- Returns a new `Timeline` containing only the selected prices.

This separation allows the same aligned input data to be optimized over different periods without changing parsing or alignment logic.

3.7 `domain.py` – Domain Data Structures

Defines the data structures exchanged across the application.

It contains:

- `Timeline`: aligned prices, common physical timestep, and market intervals.
- `BatteryConfig`: battery power limits, storage capacity, and initial energy.
- `OptimizationResult`: optimized charging, discharging, energy, and total profit.
- `DispatchRow`: market-level dispatch and cash-flow information.
- `OperationRow`: aggregated battery operation and cumulative cash flow.
- `DispatchReport`: the complete report, including metadata.

It also defines type aliases for market data, prices, timestamps, and dispatch mappings.

The module has no processing or I/O responsibilities. Its purpose is to establish explicit contracts between components.

3.8 `optimization.py` – MILP Formulation and Solver Integration

Contains the mathematical optimization implementation.

The `optimize_battery` function validates the battery configuration, prepares the timeline, builds the Pyomo model, solves it, and extracts the solution.

The model represents:

- Market-specific charging and discharging commitments, fixed over each market's trading interval.
- Battery energy at every physical timestep.
- A binary operating-mode variable for every physical timestep.
- Aggregate charging and discharging power across all markets.
- Battery energy balance, storage capacity, and power constraints.
- A profit-maximization objective based on market prices.

The model is built using Pyomo's `ConcreteModel` API and solved with HiGHS.

The solver configuration currently specifies:

- Time limit: 300 seconds.
- Relative MIP gap tolerance: 2%.

The solver status is checked before the solution is extracted. An infeasible model raises a `ValueError`; unavailable solver or unsuccessful termination raises a `RuntimeError`.

The output is an `OptimizationResult`, not a CSV or other presentation format.

3.9 `reporting.py` – Report Construction

Transforms the optimization result into report-oriented domain objects.

It creates two complementary views.

Market dispatch

One row per physical timestamp and market, containing:

- Market price.
- Charging and discharging power.
- Cash flow for that market and timestep.

Battery operation

One row per physical timestamp, containing:

- Aggregate charging and discharging power across markets.
- Stored energy at the beginning of the timestep.
- Cash flow aggregated across markets.
- Cumulative cash flow.

The reporting module also carries relevant optimization metadata into the final `DispatchReport`.

It does not write files or interact with the solver.

3.10 `csv_export.py` – Report Persistence

Serializes a `DispatchReport` into files.

It produces:

- `dispatch.csv`: market-level dispatch.
- `operation.csv`: aggregate battery operation.
- `metadata.json`: total profit, modeled period, physical timestep, battery configuration, and solver gap tolerance.

The module creates the output directory when necessary and returns the paths of the generated CSV files.

3.11 `__init__.py`

Marks the package and provides its package initialization point. It currently contains no additional application logic.

4 Data Flow and Component Interaction

Figure 2 illustrates the complete execution path, including the main processing stages.

The most important intermediate representation is `Timeline`.

Before alignment, market data is represented independently:

$$MarketData = \{market \mapsto \{timestamp \mapsto price\}\}$$

After alignment, each physical timestamp contains a price for every market:

$$TimelinePrices = \{timestamp \mapsto \{market \mapsto price\}\}$$

This representation allows the optimization module to reason about all markets using a single physical time axis.

The optimization result is then converted into a `DispatchReport`, which separates the market-level schedule from the battery-level operational view.

5 Temporal Alignment

The example workbook contains two markets:

- Half-hourly market: 30-minute trading intervals.

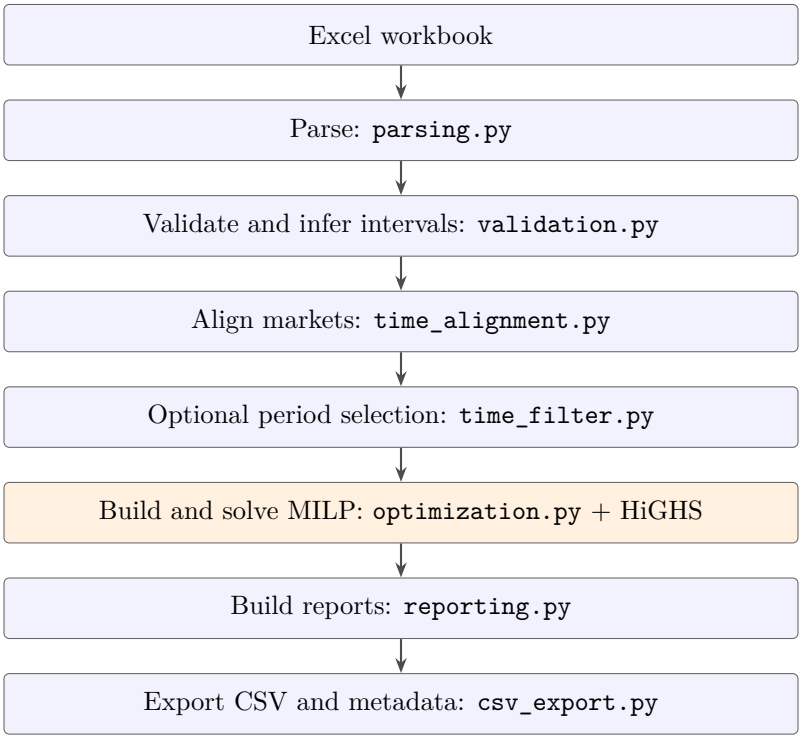


Figure 2: Sequential processing stages coordinated by `pipeline.py`.

- Hourly market: 60-minute trading intervals.

The physical timeline uses the greatest common divisor of market intervals, which is 30 minutes.

An hourly commitment therefore remains constant over two consecutive physical timesteps, while the half-hourly market can make a new commitment at each timestep.

Figure 3 illustrates this relationship.

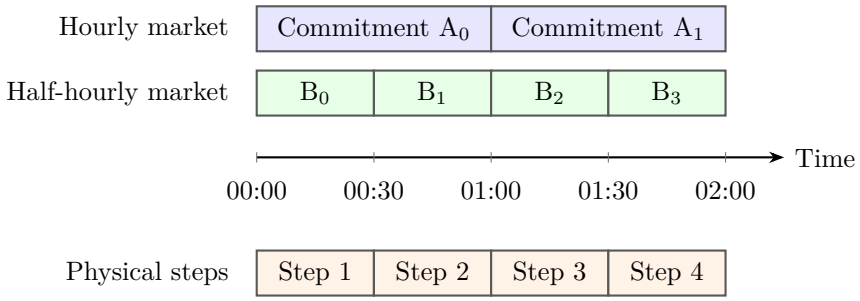


Figure 3: Hourly commitments span two half-hour physical timesteps.

This representation is essential for enforcing battery-wide constraints. Charging and discharging decisions from different markets must be considered together at every physical timestep.

Without a shared timeline, the optimization model could incorrectly treat each market as if it had independent access to the battery.

6 Testing Strategy

The test suite is organized around the application's modules.

Test module	Main responsibility covered
`test_cli.py`	Command-line arguments, execution, and error handling
`test_parsing.py`	Excel parsing and invalid input handling
`test_validation.py`	Market interval inference and timestamp validation
`test_time_alignment.py`	Grid normalization, common coverage, and price alignment
`test_time_filter.py`	Period selection and boundary validation
`test_optimization.py`	MILP constraints, feasibility, and optimization results
`test_reporting.py`	Dispatch and operation report construction
`test_csv_export.py`	CSV and metadata serialization
`test_pipeline.py`	End-to-end workflow
`test_pipeline_stages.py`	Pipeline stage orchestration

This organization makes it possible to test individual transformations independently while also validating that the complete workflow behaves correctly.

The project also configures Ruff for linting and import checks, and mypy for static type checking.

7 Design Characteristics and Trade-offs

The architecture separates concerns into modules with explicit inputs and outputs. This makes the system easier to reason about and allows individual stages to be tested independently.

Some important design decisions are worth highlighting.

7.1 Common Physical Timeline

All market decisions are evaluated against the same physical timeline. This is necessary to enforce shared battery constraints when markets have different trading intervals.

7.2 Market Interval Commitments

Charging and discharging decisions are indexed by market interval rather than by physical timestep. A commitment in a longer-duration market therefore remains fixed throughout that interval.

7.3 Optimization and Reporting Separation

The optimization module is responsible for finding a feasible schedule and calculating its objective value. The reporting module independently transforms that schedule into operational and financial records.

7.4 Explicit Domain Contracts

The domain module defines the structures exchanged between processing stages. This reduces coupling between modules and makes their responsibilities clearer.

7.5 Configurable Battery Parameters

The optimization function accepts a `BatteryConfig`, allowing library callers to change battery power limits, capacity, and initial energy without modifying the optimization formulation.

8 Current Scope and Limitations

The application is an optimization model, not a complete battery trading or control system.

Its main assumptions are:

- Historical market prices are known over the entire optimization horizon (perfect foresight).
- The battery is a price taker and does not influence market prices.
- Charging and discharging efficiency are both 100%.
- Battery degradation and transaction costs are not modeled.
- Charging and discharging cannot happen simultaneously.
- The final stored energy is not constrained to equal the initial energy and has no explicit terminal value.
- Market commitments are treated as jointly optimizable, with perfect knowledge of all market prices.

Consequently, the reported profit represents the optimal trading outcome under these assumptions. It should not be interpreted as a forecast or a guarantee of real-world returns.

9 Summary

The project is structured around a clear separation between data ingestion, temporal modeling, optimization, and reporting.

The most distinctive architectural element is the common physical timeline, which reconciles markets with different trading intervals and provides the foundation for enforcing shared battery constraints.

The pipeline coordinates the workflow, domain objects define the contracts between stages, and the optimization module remains isolated from input and output concerns.

This structure keeps the mathematical model focused while allowing data handling, reporting, and execution to evolve independently.

Project repository